

ENTREGA PRIMERA EVALUACIÓN

ANÁLISIS Y DISEÑO DE SOFTWARE

Planificación, requerimientos, modelado UML, base de datos y mockup funcional

NOMBRE / INTEGRANTES: _____

CARRERA: Analista en Informática, Ing en Informática e Ing en Ciberseguridad

ASIGNATURA: Proyecto Integrado

PROFESOR: Jorge Cortés

FECHA: Próxima semana – según fecha habilitada en plataforma

Documento de instrucciones y rúbrica de evaluación

1. Objetivo de la entrega

La entrega tiene por objetivo consolidar el análisis y diseño técnico de la solución antes de avanzar a etapas de integración y desarrollo completo. Se evaluará la coherencia entre la planificación del proyecto, los requisitos, los modelos UML, el diseño de datos y el prototipo funcional.

Criterio transversal de coherencia

Los artefactos no deben presentarse como piezas aisladas. Debe existir trazabilidad entre requerimientos, casos de uso, clases, entidades de base de datos y pantallas del mockup, cuando corresponda.

2. Condiciones generales de entrega

- **Formato del informe:** utilizar el presente formato institucional y mantener una estructura ordenada por secciones.
- **Repositorio Git:** incorporar el enlace al repositorio y asegurar acceso al docente. El repositorio debe contener el mockup funcional y, de ser posible, los archivos editables de los diagramas.
- **Legibilidad:** todos los diagramas deben insertarse con resolución suficiente para leer actores, relaciones, atributos, claves y cardinalidades.
- **Identificación:** usar códigos consistentes para requisitos y casos de uso, por ejemplo RF-01, RNF-01, CU-01, CU-02, etc.
- **Evidencias:** incluir capturas del Planner/Jira y del mockup, además de los enlaces correspondientes.
- **Consistencia:** nombres de módulos, entidades, actores y funciones deben mantenerse homogéneos en todos los artefactos.

3. Alcance del mockup funcional

En esta etapa el prototipo se evaluará por navegación, fluidez de interfaces, distribución de elementos, validación de campos y uso apropiado de librerías externas. No se exige integración con base de datos ni consumo de API para esta entrega.

4. Entregables obligatorios

4.1 Planner / Jira

Se debe presentar la planificación operativa del proyecto en Planner o Jira. La evidencia debe permitir comprender qué se construirá, quién es responsable y cuál es el estado de avance.

- Backlog del proyecto organizado y priorizado.
- Épicas y/o módulos principales del sistema.
- Historias de usuario, tareas o actividades asociadas a cada épica.
- Responsables asignados por tarea o historia.
- Estados de trabajo claramente diferenciados, por ejemplo: Por hacer, En desarrollo, En revisión y Finalizado.
- Fechas, hitos o sprint cuando la herramienta y la metodología utilizada lo permitan.
- Prioridad y dependencias relevantes entre tareas.
- Captura del tablero y enlace directo al Planner/Jira cuando sea posible.

4.2 Diagramas de requerimientos

Se deben presentar diagramas que permitan visualizar y relacionar los requerimientos del sistema. La notación puede ajustarse a la herramienta utilizada, pero debe ser clara y trazable.

- Identificar requerimientos funcionales (RF) y no funcionales (RNF) mediante códigos únicos.
- Relacionar requerimientos con actores, módulos o procesos del sistema.
- Mostrar dependencias, agrupaciones o relaciones entre requisitos cuando corresponda.
- Mantener concordancia con el diagrama general de casos de uso y con los casos de uso específicos.
- Evitar requerimientos ambiguos; cada requisito debe representar una necesidad verificable.

ID	Requerimiento	Actor	Módulo	Trazabilidad
RF-01	Autenticación de usuario	Usuario	Acceso / Seguridad	CU-01
RF-02	Registrar información	Usuario autorizado	Módulo principal	CU-02
RNF-01	Validar campos obligatorios	Sistema	Calidad / UX	CU-01 a CU-10

5. Modelado de casos de uso

5.1 Diagrama de caso de uso general

Debe existir un diagrama general que represente el sistema completo a nivel funcional, identificando sus principales actores y servicios.

- Límite o frontera del sistema claramente identificado.
- Actores principales y secundarios correctamente nombrados.
- Casos de uso principales del sistema, sin detallar flujos internos.
- Relaciones actor–caso de uso y uso adecuado de «include», «extend» o generalización cuando corresponda.
- Correspondencia con los requerimientos funcionales definidos.

5.2 Casos de uso específicos con especificación – mínimo 10

Se deben entregar como mínimo diez (10) casos de uso específicos. Cada caso debe incluir su diagrama específico y una ficha de especificación textual completa.

Condición mínima

Se exigen al menos 10 casos de uso específicos. Si se presentan menos de 10, el criterio correspondiente quedará en nivel “Insuficiente”, con un máximo de 7/20 puntos.

Ficha mínima para cada caso de uso

Campo	Contenido requerido
ID	CU-01
Nombre	Nombre funcional del caso de uso
Objetivo	Resultado que debe lograr el usuario o sistema
Actor principal	Actor que inicia o ejecuta el caso
Actores secundarios	Actores o sistemas de apoyo, si aplica
Precondiciones	Condiciones que deben cumplirse antes de iniciar
Disparador	Evento que inicia el caso de uso
Flujo principal	Secuencia normal de pasos numerados
Flujos alternativos	Variantes válidas del flujo principal
Excepciones	Errores o condiciones que impiden completar el proceso
Postcondiciones	Estado esperado al finalizar
Reglas / requisitos relacionados	RF, RNF o reglas de negocio vinculadas

6. Diseño orientado a objetos y datos

6.1 Diagrama de clases

El diagrama de clases debe representar la estructura lógica del software y evidenciar cómo se organizan las responsabilidades principales del sistema.

- Clases del dominio y/o componentes relevantes del sistema.
- Atributos significativos con tipo de dato cuando corresponda.
- Métodos u operaciones principales relacionados con las responsabilidades de la clase.
- Visibilidad de atributos y métodos (+, -, #) cuando se utilice UML formal.
- Asociaciones con multiplicidades.
- Herencia, composición o agregación solamente cuando exista justificación en el diseño.
- Nombres coherentes con los casos de uso y con el modelo de datos.

6.2 DER – Diagrama Entidad–Relación para MySQL

El DER debe mostrar cómo se almacenará la información persistente del sistema en MySQL. Debe ser consistente con el script SQL entregado.

- Entidades o tablas correctamente nombradas.
- Clave primaria (PK) definida para cada tabla que lo requiera.
- Claves foráneas (FK) y relaciones entre tablas.
- Cardinalidades 1:1, 1:N y N:M representadas correctamente.
- Tablas asociativas para resolver relaciones N:M cuando corresponda.
- Atributos/campos relevantes y tipos de datos compatibles con MySQL.
- Restricciones básicas y diseño orientado a evitar redundancias innecesarias.

Validación de consistencia

Toda FK representada en el DER debe existir en el script SQL. Del mismo modo, las tablas creadas en el script deben corresponder al modelo presentado en el informe.

7. Implementación inicial

7.1 Script de base de datos MySQL

Se debe entregar un archivo .sql ejecutable que permita crear la estructura de base de datos diseñada.

- Creación de la base de datos o selección explícita del esquema.
- Sentencias CREATE TABLE ordenadas correctamente.
- Definición de PK y FK.
- Tipos de datos apropiados para los campos.
- Restricciones como NOT NULL, UNIQUE, DEFAULT u otras cuando corresponda.
- Índices adicionales si el diseño los justifica.
- Datos de prueba (INSERT) opcionales, pero recomendados para demostrar estructura y relaciones.
- El script debe ejecutarse sin errores de sintaxis ni errores de integridad referencial.

7.2 Template / Mockup funcional en Git

El prototipo debe estar implementado y versionado en un repositorio Git. Debe permitir recorrer las principales pantallas y demostrar el diseño de interacción previsto.

- Navegación funcional entre pantallas o vistas principales.
- Flujo de interfaz comprensible y consistente.
- Distribución adecuada de elementos visuales y formularios.
- Validación de campos obligatorios, formatos y mensajes básicos al usuario.
- Uso de librerías externas cuando aporten al diseño o funcionalidad.
- Código organizado y almacenado en el repositorio Git.
- README con descripción del proyecto, tecnologías utilizadas e instrucciones para ejecutar el mockup.
- Enlace del repositorio incorporado en el informe.

Importante

Para esta entrega no se evaluará conexión con base de datos ni consumo de API. El foco está en la experiencia de navegación, consistencia visual, validaciones y estructura del prototipo.

8. Rúbrica de evaluación

Puntaje total: 100 puntos. La calificación se obtiene aplicando la escala institucional vigente. La evidencia debe ser verificable, legible y coherente con el proyecto presentado.

Criterio	Ptos.	Evidencia evaluada	Niveles de desempeño
Planner / Jira	15	Backlog, épicas/tareas, responsables, estados, prioridad, hitos y evidencia del tablero.	Excelente: 15 pts. Planificación completa, priorizada, con responsables, estados, hitos y trazabilidad clara del trabajo. Logrado: 11 pts. Planificación adecuada y utilizable, con pequeñas omisiones de prioridad, fechas o dependencias. En desarrollo: 7 pts. Tablero parcialmente construido; faltan responsables, estados consistentes o detalle suficiente del trabajo. Insuficiente: 0–4 pts. Evidencia mínima, desordenada o inexistente; no permite comprender la planificación.
Diagramas de requerimientos	10	RF/RNF identificados, relación con actores/módulos y trazabilidad hacia funcionalidades.	Excelente: 10 pts. Requisitos claros, identificados, relacionados y completamente coherentes con el resto de los modelos. Logrado: 7 pts. Requisitos correctos con algunas relaciones o trazas faltantes. En desarrollo: 5 pts. Requisitos incompletos, ambiguos o con trazabilidad limitada. Insuficiente: 0–3 pts. Diagrama ausente, ilegible o sin relación verificable con el sistema.
Caso de uso general	10	Actores, frontera del sistema, casos de uso principales y relaciones UML.	Excelente: 10 pts. Representa correctamente el alcance global, actores y funcionalidades, usando relaciones UML coherentes. Logrado: 7 pts. Modelo global correcto con omisiones menores o relaciones mejorables. En desarrollo: 5 pts. Modelo parcial; faltan actores, funciones o claridad en el alcance. Insuficiente: 0–3 pts. Diagrama incorrecto, inconsistente o no presentado.
Casos de uso específicos + especificaciones	20	Mínimo 10 casos específicos, diagramas y fichas con flujos, pre/postcondiciones, excepciones y trazabilidad.	Excelente: 20 pts. Se presentan 10 o más casos completos, coherentes, detallados y trazables con requisitos y actores. Logrado: 15 pts. Se presentan 10 casos adecuados, con omisiones menores en flujos alternativos, excepciones o reglas. En desarrollo: 10 pts. Casos incompletos, poco detallados o con inconsistencias; pueden faltar elementos de especificación. Insuficiente: 0–7 pts. Menos de 10 casos, especificaciones muy deficientes o evidencia insuficiente.

Criterio	Ptos.	Evidencia evaluada	Niveles de desempeño
Diagrama de clases	15	Clases, atributos, métodos, relaciones, multiplicidades y coherencia con funcionalidades.	Excelente: 15 pts. Modelo estructural completo, consistente, con responsabilidades y relaciones UML correctamente definidas. Logrado: 11 pts. Modelo adecuado con pequeñas omisiones de métodos, multiplicidades o relaciones. En desarrollo: 7 pts. Clases básicas presentes, pero con diseño incompleto o inconsistencias relevantes. Insuficiente: 0–4 pts. Modelo ausente, ilegible o conceptualmente incorrecto.
DER MySQL	10	Entidades/tablas, PK, FK, cardinalidades, campos y coherencia relacional.	Excelente: 10 pts. DER completo y consistente, con relaciones y claves correctamente definidas y sin redundancias evidentes. Logrado: 7 pts. Modelo correcto con errores menores en cardinalidades, tipos o restricciones. En desarrollo: 5 pts. Modelo parcial con relaciones o claves incompletas. Insuficiente: 0–3 pts. DER incorrecto, inconsistente con el sistema o no presentado.
Script SQL	10	Archivo .sql ejecutable, tablas, PK/FK, tipos, restricciones y consistencia con el DER.	Excelente: 10 pts. Script ordenado, ejecutable sin errores y totalmente consistente con el DER. Logrado: 7 pts. Script funcional con errores menores o restricciones mejorables. En desarrollo: 5 pts. Script incompleto o requiere correcciones para ejecutarse correctamente. Insuficiente: 0–3 pts. Script no ejecuta, no corresponde al DER o no se entrega.
Mockup funcional + Git	10	Navegación, fluidez, distribución visual, validaciones, librerías, código versionado y README.	Excelente: 10 pts. Prototipo navegable, consistente, validado, bien organizado y disponible en un repositorio accesible y documentado. Logrado: 7 pts. Prototipo funcional con pequeñas fallas de navegación, validación o documentación. En desarrollo: 5 pts. Prototipo parcial; navegación limitada, validaciones incompletas o repositorio poco documentado. Insuficiente: 0–3 pts. Mockup no funcional, repositorio inaccesible o evidencia insuficiente.



APRENDER HACIENDO A LO LARGO DE LA VIDA

ACREDITADO

7[~]AÑOS
MÁXIMA
EXCELENCIA



INSTITUTO PROFESIONAL INACAP

ACREDITADO • NIVEL DE EXCELENCIA

- Gestión Institucional • Docencia de Pregrado
- Vinculación con el Medio
- Próxima Acreditación Septiembre 2030

www.inacap.cl